

Lab 3: Coding and Maximizing a Likelihood

SOCIOL 723, Social Statistics II, Thursday, September 10

Wenhao Jiang

Table of contents

1	Goals	1
2	The likelihood as an R function	2
2.1	A binomial likelihood	2
2.2	Why we work in logs	2
3	Maximizing: the Poisson case	3
3.1	Standard errors from the Hessian	4
4	Least squares falls out of the normal likelihood	4
4.1	The likelihood surface	6
5	What non-identification looks like	7
6	The tests, by hand	8
6.1	One coefficient: the z you already have	9
6.2	A block of restrictions	9
7	Exercises	11
8	Session information	12

1 Goals

1. Write a log-likelihood as an R function and plot it.
2. Maximize it numerically with `optim()`, and check the answer against the closed form.
3. Get standard errors from the Hessian, and check them against the second derivative you can work out by hand.
4. Watch least squares fall out of the normal likelihood, digit for digit.
5. See what non-identification looks like.
6. Compute the Wald and likelihood ratio tests by hand.

```
library(dplyr)
library(ggplot2)
gss <- readRDS("../Data/gss_earnings.rds")
```

2 The likelihood as an R function

2.1 A binomial likelihood

Ten draws, seven blue marbles. The likelihood is $\mathcal{L}(\theta) = \binom{10}{7}\theta^7(1-\theta)^3$.

```
## dbinom() with x fixed at 7 and prob left free: the likelihood of theta
lik <- function(theta) dbinom(x = 7, size = 10, prob = theta)
curve(lik, from = 0, to = 1, n = 400, col = "navy", lwd = 2,
      xlab = expression(theta), ylab = expression(L(theta)))
abline(v = 0.7, col = "firebrick", lty = 2) # the peak: 7/10
```

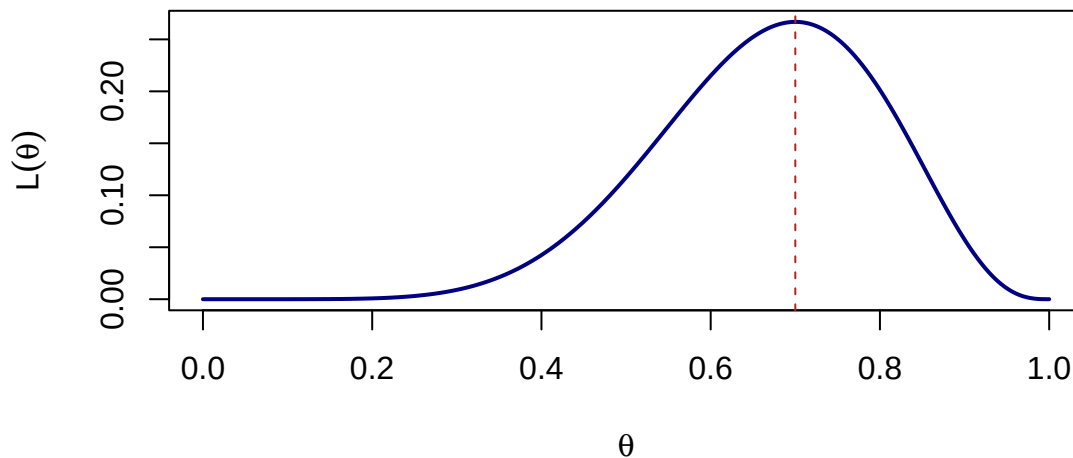


Figure 1: Binomial likelihood, 7 successes in 10 trials.

Note what `dbinom()` is doing: it is the *same function* whether you use it to compute a probability (fix **prob**, vary **x**) or a likelihood (fix **x**, vary **prob**). Only your reading of it changes.

2.2 Why we work in logs

[illegible]

```
## the log-likelihood is a sum of 500 logs: perfectly well behaved
sum(dbinom(y, size = 1, prob = 0.3, log = TRUE))
#> [1] -324.07
```

Tip

Every `d*`() function in R takes `log = TRUE`. Use it. Computing `log(dnorm(x))` instead of `dnorm(x, log = TRUE)` is both slower and less accurate.

3 Maximizing: the Poisson case

The GSS variable `chilids` counts a respondent's children. Start with the simplest possible model, $Y_i \sim \text{Poisson}(\theta)$ with no covariates.

```
y <- gss$chilids

## the NEGATIVE log-likelihood: optim() minimizes, so hand it -l(theta).
## dpois(..., log = TRUE) is log f(y_i | theta); summing over i gives l(theta)
nll_pois <- function(theta) -sum(dpois(y, lambda = theta, log = TRUE))

## BFGS: a Newton-type search that builds up the curvature as it goes
## hessian = TRUE returns -l'' at the optimum
fit <- optim(par = 1, fn = nll_pois, method = "BFGS",
            hessian = TRUE)

## convergence == 0 means optim() is satisfied it found the minimum
c(mle_numeric = fit$par, closed_form = mean(y), converged = fit$convergence)
#> mle_numeric closed_form converged
#>      1.6044      1.6044      0.0000

## evaluate l on a grid of theta values; the minus sign turns -l back into l
th <- seq(1.2, 2.1, length.out = 300)
plot(th, -sapply(th, nll_pois), type = "l", col = "navy", lwd = 2,
     xlab = expression(theta), ylab = expression(ell(theta)))
abline(v = mean(y), col = "firebrick", lty = 2) # the closed-form MLE, ybar
```

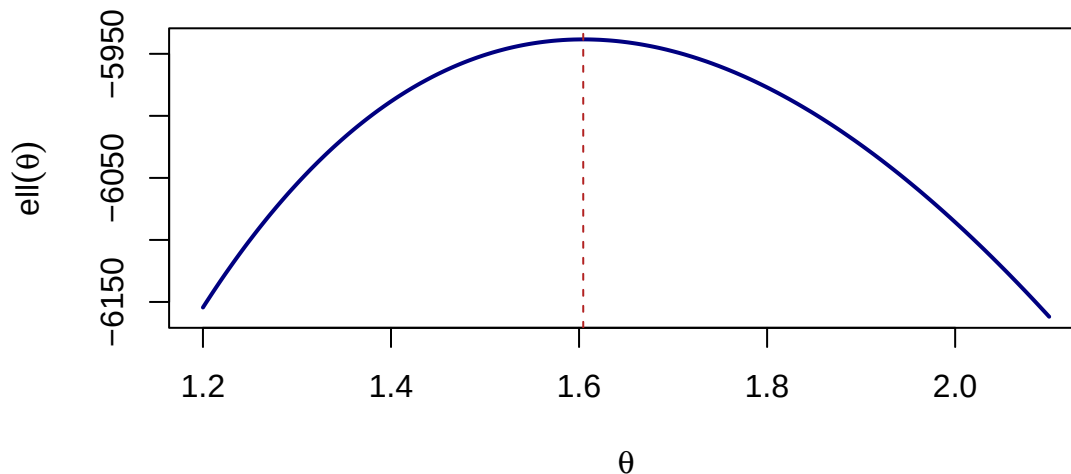


Figure 2: Poisson log-likelihood for the number of children, GSS.

3.1 Standard errors from the Hessian

We minimized $-\ell$, so the Hessian `optim()` returns is already $-\partial^2\ell/\partial\theta^2$ at $\hat{\theta}$: the observed information. Its inverse is the variance. By hand, from lecture, $-\ell''(\theta) = \sum_i y_i/\theta^2$, which at $\hat{\theta} = \bar{y}$ is $n/\bar{y}$.

```
## optim() minimized -l, so its Hessian is -l''(theta_hat), the observed
## information; variance = 1 / information, SE = its square root
se_hessian <- sqrt(1 / fit$hessian)
se_by_hand <- sqrt(mean(y) / length(y)) # 1 / sqrt(n / ybar)

c(hessian = as.numeric(se_hessian), by_hand = se_by_hand)
#> hessian by_hand
#> 0.021383 0.021383
```

They agree, as they must: the numerical second derivative is approximating the one you just wrote down.

```
## Wald interval: the peak plus or minus 1.96 SEs
c(estimate = mean(y),
  lower = mean(y) - 1.96 * se_by_hand,
  upper = mean(y) + 1.96 * se_by_hand)
#> estimate lower upper
#> 1.6044 1.5625 1.6464
```

4 Least squares falls out of the normal likelihood

Lecture’s “Cashing the Hook”: assume a linear CEF and normal errors, and the MLE of the line is the least-squares line. Check it. Three parameters, (b_0, b_1, σ) , for `llearn` on `educ`.

```

z <- gss$lnearn; x <- gss$educ

## three parameters in one vector: par = (b0, b1, log sigma)
nll_norm <- function(par) {
  b0 <- par[1]; b1 <- par[2]; sigma <- exp(par[3]) # log-parameterize sigma > 0
  ## y_i is normal with mean b0 + b1 x_i and sd sigma; sum the log densities
  -sum(dnorm(z, mean = b0 + b1 * x, sd = sigma, log = TRUE))
}

## BFGS: a Newton-type search that builds up the curvature as it goes.
## Start at c(8, 0.1, 0): roughly mean(lnearn), a small slope, log(sigma) = 0
fit2 <- optim(c(8, 0.1, 0), nll_norm, method = "BFGS", hessian = TRUE)
m_ols <- lm(lnearn ~ educ, data = gss)

## the MLE line (first two parameters) against the least-squares line
round(rbind(mle = fit2$par[1:2], ols = coef(m_ols)), 6)
#>      (Intercept)      educ
#> mle          8.2307 0.11873
#> ols          8.2307 0.11873

```

Digit for digit. Now σ :

```

e <- residuals(m_ols); n <- length(e)
c(sigma_mle      = exp(fit2$par[3]),          # undo the log-parameterization
  divide_by_n    = sqrt(sum(e^2) / n),        # lecture's MLE formula
  divide_by_n_2  = sqrt(sum(e^2) / (n - 2))) # lm's s
#>      sigma_mle  divide_by_n divide_by_n_2
#>      0.87191    0.87191    0.87216

```

i Note

Two things worth noticing.

1. We optimized over $\log \sigma$ rather than σ . Constrained parameters (variances, probabilities, rates) should be transformed to the whole real line before handing them to an unconstrained optimizer; $\hat{\sigma} = \exp(\widehat{\log \sigma})$ is still the MLE.
2. The MLE of σ^2 divides by n , not $n - 2$: it is *biased*, exactly as lecture derived. With $n = 3,509$ the difference is in the fourth decimal, which is the other half of the lesson: the bias vanishes as n grows.

The standard error of the slope from the Hessian is the OLS standard error with $\hat{\sigma}^2$ in place of s^2 , so the two differ by the factor $\sqrt{(n-2)/n}$:

```

## invert the whole 3 x 3 Hessian, then read off the slope's diagonal entry
c(hessian = sqrt(solve(fit2$hessian)[2, 2])),

```

```

lm      = coef(summary(m_ols))["educ", "Std. Error"],
lm_x_factor = coef(summary(m_ols))["educ", "Std. Error"] * sqrt((n - 2) / n))
#>      hessian      lm lm_x_factor
#> 0.0052110 0.0052125 0.0052110
## the last line rescales lm's SE from  $s^2$  to  $\sigma_{\hat{}}^2$ 

```

4.1 The likelihood surface

```

## an 80 x 80 grid of (b0, b1) pairs around the OLS estimates
b0_grid <- seq(8.05, 8.40, length.out = 80)
b1_grid <- seq(0.105, 0.132, length.out = 80)
g <- expand.grid(b0 = b0_grid, b1 = b1_grid)
s_hat <- exp(fit2$par[3]) # sigma held at its MLE throughout

## l(b0, b1) at every grid point: the surface, 6,400 evaluations
g$ll <- mapply(function(a, b) sum(dnorm(z, a + b * x, s_hat, log = TRUE)), g$b0, g$b1)

## filled contours of l, with the OLS point drawn on top
ggplot(g, aes(b0, b1, z = ll)) +
  geom_contour_filled(bins = 12) +
  geom_point(aes(x = coef(m_ols)[1], y = coef(m_ols)[2]),
             colour = "white", size = 2, inherit.aes = FALSE) +
  labs(x = expression(b[0]), y = expression(b[1]), fill = expression(e11)) +
  theme_minimal(base_size = 9) + theme(legend.key.size = unit(0.3, "cm"))

```

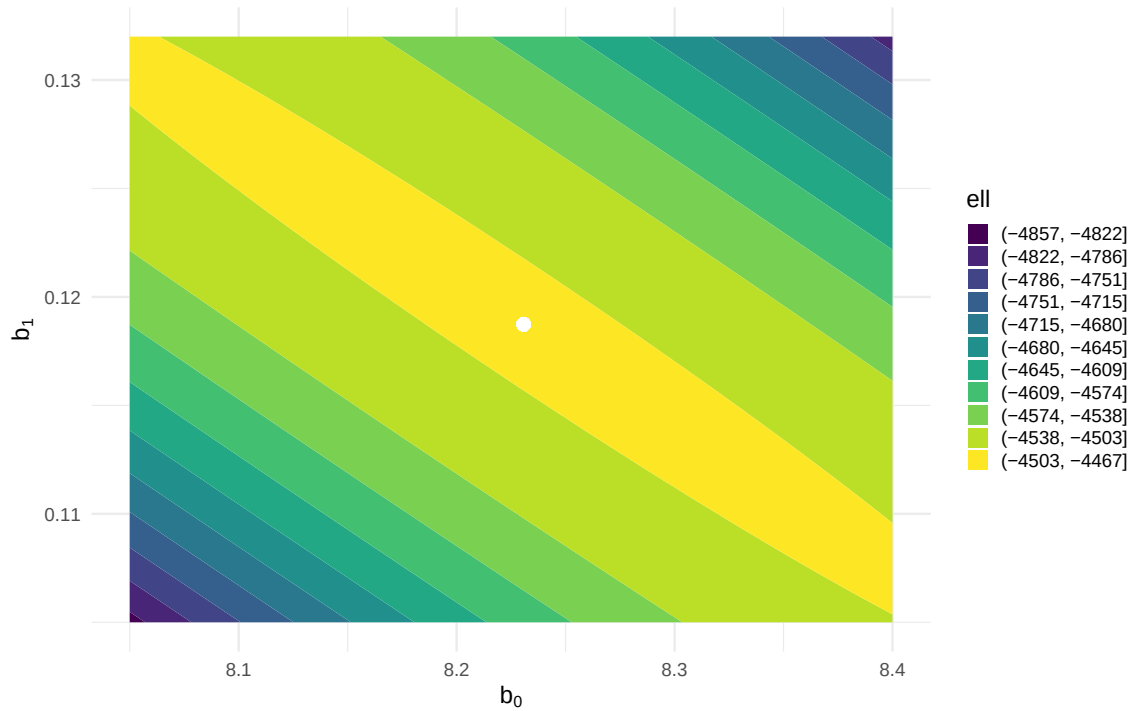


Figure 3: Log-likelihood surface for the normal linear model, σ fixed at its MLE. The contours are ellipses centred on the OLS estimates, tilted because the intercept and slope trade off.

5 What non-identification looks like

Suppose we (foolishly) parameterize the mean as $\theta_1 + \theta_2$. The data can only speak to the sum.

```
set.seed(1)
w <- rnorm(200, mean = 3, sd = 1) # 200 draws; the model calls the mean theta1 + theta2

## only the SUM theta1 + theta2 enters the density, so the data cannot separate them
nll_bad <- function(par) -sum(dnorm(w, mean = par[1] + par[2], sd = 1, log = TRUE))

## 30 random starting values, one optimization from each
starts <- matrix(rnorm(60, sd = 3), ncol = 2)
res <- t(apply(starts, 1, function(s) optim(s, nll_bad)$par))
head(round(res, 3), 4) # different starts, different "answers"
#>      [,1] [,2]
#> [1,]  4.047 -1.011
#> [2,]  6.200 -3.164
#> [3,]  4.525 -1.490
#> [4,] -0.053  3.089

plot(res, col = "navy", pch = 19, cex = .7,
      xlab = expression(hat(theta)[1]), ylab = expression(hat(theta)[2]))
```

```
## every estimate satisfies theta1 + theta2 = mean(w): the ridge
abline(a = mean(w), b = -1, col = "firebrick", lwd = 1.5)
```

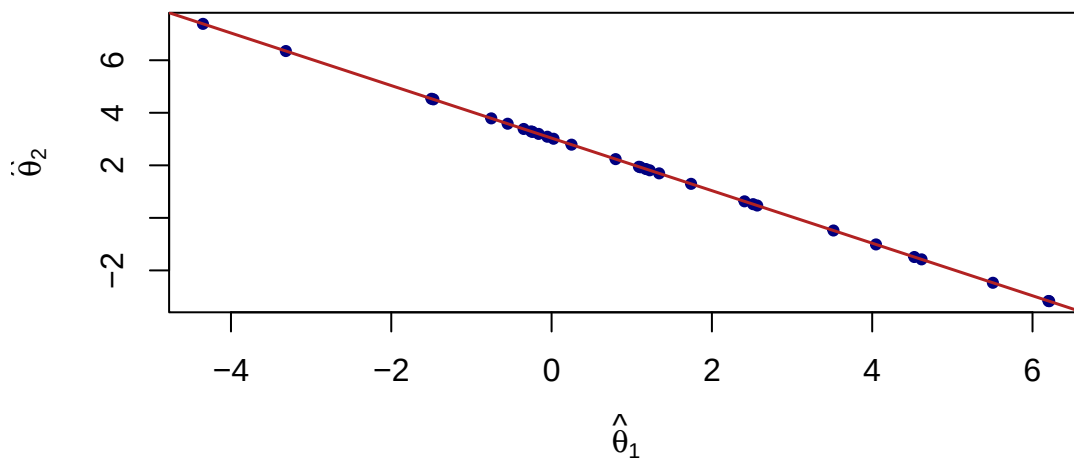


Figure 4: Estimates from 30 different starting values. They lie on the ridge $\theta_1 + \theta_2 = \bar{w}$.

The Hessian is singular, so there are no standard errors:

```
## the Hessian of a ridge: along the ridge the curvature is zero
H <- optim(c(0, 0), nll_bad, hessian = TRUE)$hessian
H
#>      [,1] [,2]
#> [1,]  200  200
#> [2,]  200  200
qr(H)$rank      # 1, not 2
#> [1] 1
try(solve(H), silent = TRUE) |> class() # "try-error": no inverse, no SEs
#> [1] "try-error"
```

! Important

This is the likelihood analogue of perfect collinearity. **The lesson:** always run your optimizer from several starting values. If they disagree, you have an identification problem, not a numerical one.

6 The tests, by hand

The outcome is whether the respondent holds a bachelor's degree or more (**ba**), and the model is a logit fitted by `glm(..., family = binomial)`. Week 4 opens up what that model is; today it is a black box that hands us a log-likelihood, and every test below is read off that curve.

6.1 One coefficient: the z you already have

Lecture started from the standard error. `summary()` prints a z for every coefficient: the Wald test. Take black.

```
## the unrestricted model; summary() divides each coefficient by its SE
m_full <- glm(ba ~ pareduc + wordsum + female + black,
              data = gss, family = binomial)
z <- coef(summary(m_full))["black", "z value"]
c(z = z, z_squared = z^2) # z^2 is the Wald statistic W, compared with chi2_1
#>      z z_squared
#> -1.0417  1.0852
```

A null hypothesis is a smaller model: $\beta_{\text{black}} = 0$ is the logit without black. Fit it and measure the drop in the log-likelihood.

```
## the restricted model: the null beta_black = 0, imposed by leaving black out
m_no_bl <- glm(ba ~ pareduc + wordsum + female, data = gss, family = binomial)

## logLik() is l at each model's peak; their difference is the vertical ruler
drop <- as.numeric(logLik(m_full) - logLik(m_no_bl))
c(drop = drop, LR = 2 * drop) # LR = twice the drop
#>  drop    LR
#> 0.5454 1.0908

c(Wald = z^2, LR = 2 * drop)
#>  Wald    LR
#> 1.0852 1.0908
```

Two rulers, one distance, and here they agree to two decimals: the null is about one standard error from the peak, where the log-likelihood is as good as a parabola. Both are compared with χ^2_1 , whose 5% critical value is $1.96^2 = 3.84$, so neither rejects.

```
## R's own LR test: "Deviance" is LR, "Pr(>Chi)" its chi2_1 p-value
anova(m_no_bl, m_full, test = "Chisq")
#> Analysis of Deviance Table
#>
#> Model 1: ba ~ pareduc + wordsum + female
#> Model 2: ba ~ pareduc + wordsum + female + black
#>   Resid. Df Resid. Dev Df Deviance Pr(>Chi)
#> 1      3505      3960
#> 2      3504      3959  1      1.09      0.3
```

6.2 A block of restrictions

A one-coefficient test asks about one number. Many real questions are about a *group* of coefficients at once, and a categorical variable is the standard case. The GSS records nine census regions.

Does region matter for holding a degree? That question is not about any single dummy: with New England as the reference, the eight coefficients compare each other region with New England, and none of them alone is “region.”

```
## the unrestricted model adds the region factor: eight dummies, New England omitted
m_reg <- glm(ba ~ pareduc + wordsum + female + black + region,
             data = gss, family = binomial)

## the eight one-at-a-time Wald tests
round(coef(summary(m_reg))[grep("^region", names(coef(m_reg))), ], 3)
#>
#> Estimate Std. Error z value Pr(>|z|)
#> regionMiddle Atlantic      0.206      0.199   1.038   0.299
#> regionE. North Central    -0.336      0.187  -1.793   0.073
#> regionW. North Central    -0.546      0.218  -2.499   0.012
#> regionSouth Atlantic     -0.302      0.186  -1.622   0.105
#> regionE. South Central    -0.322      0.228  -1.408   0.159
#> regionW. South Central    -0.441      0.208  -2.120   0.034
#> regionMountain           -0.350      0.211  -1.663   0.096
#> regionPacific             -0.195      0.196  -0.994   0.320
```

Two of the eight are significant at 5%, three more are near it, and the rest are not. Read one at a time, this is a muddle. The right question is the joint one: H_0 : all eight region coefficients are zero. That is $q = 8$ restrictions, so the reference distribution is χ^2_8 , whose 5% cut-off is 15.5. The restricted model is `m_full`, which has no region terms.

```
## ---- likelihood ratio: twice the drop between the two peaks ----
LR <- 2 * as.numeric(logLik(m_reg) - logLik(m_full))

## ---- Wald: the eight coefficients against their 8 x 8 variance block ----
## W = b_R' V_R^{-1} b_R is the eight-dimensional version of (gap / SE)^2
idx <- grep("^region", names(coef(m_reg)))
b_R <- coef(m_reg)[idx]
V_R <- vcov(m_reg)[idx, idx]
W <- as.numeric(t(b_R) %*% solve(V_R) %*% b_R)

## each is compared with chi2_8, whose 5% cut-off is 15.5
round(c(Wald = W, LR = LR), 2)
#> Wald LR
#> 27.71 27.84
sprintf("p-values: %.1e %.1e", pchisq(W, 8, lower.tail = FALSE),
        pchisq(LR, 8, lower.tail = FALSE))
#> [1] "p-values: 5.3e-04 5.1e-04"
```

Both rulers say about 28 on 8 degrees of freedom: region matters, at $p \approx 0.0005$, even though six

of the eight dummies could not say so on their own. They agree to a decimal because $n = 3,509$ makes the log-likelihood a parabola in every direction at once. Two things to carry away. A block test answers a question no single z can. And the block test does not care which region is the reference: change the omitted category and every one of the eight z 's changes, but W and LR do not, because the restricted model is the same model either way.

```
## R's LR test for the block: Df = 8, Deviance = LR
anova(m_full, m_reg, test = "Chisq")
#> Analysis of Deviance Table
#>
#> Model 1: ba ~ pareduc + wordsum + female + black
#> Model 2: ba ~ pareduc + wordsum + female + black + region
#>   Resid. Df Resid. Dev Df Deviance Pr(>Chi)
#> 1      3504      3959
#> 2      3496      3931  8      27.8  0.00051 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

7 Exercises

1. **Bernoulli information.** Simulate $n = 200$ draws from $\text{Bernoulli}(0.3)$. Compute the MLE, the observed information from `optim(hessian = TRUE)`, and the hand-derived $-\ell''(\hat{\theta}) = n/[\hat{\theta}(1 - \hat{\theta})]$. Confirm the two standard errors agree.
2. **Curvature is information.** For $n \in \{25, 100, 400, 1600\}$, simulate $\text{Poisson}(2)$ data and plot the four log-likelihoods on the same axes, each shifted to peak at zero. Describe what happens to the width of the peak, and connect it to $\mathcal{I}_n = n\mathcal{I}_1$.
3. **Newton–Raphson by hand.** Lecture's picture used one Poisson observation, $y = 5$, so $\ell(\theta) = 5 \log \theta - \theta$, $S(\theta) = 5/\theta - 1$, and $\ell''(\theta) = -5/\theta^2$. Starting from $\theta^{(0)} = 1.5$, iterate $\theta^{(t+1)} = \theta^{(t)} - S(\theta^{(t)})/\ell''(\theta^{(t)})$ and print, at each step, the guess, the slope, $-\ell''$, and the step. Reproduce the slide's table, and say in one sentence why the step shrinks near the peak even though $-\ell''$ is smallest there.
4. **Not unbiased, but consistent.** Simulate $n = 20$ observations from $Y_i = 1 + 0.5X_i + \epsilon_i$ with $\epsilon_i \sim N(0, 1)$, fit the line by OLS, and compute $\hat{\sigma}_{\text{MLE}}^2 = \sum_i \hat{\epsilon}_i^2/n$. Over 5,000 replications, compare its mean with $\sigma^2 = 1$ and with $(n - 2)/n$. Repeat at $n = 200$. What happens to the bias?
5. **Where the z -test fails.** Twenty Bernoulli trials, zero successes. Compute $\hat{\theta}$, its standard error $\sqrt{\hat{\theta}(1 - \hat{\theta})/n}$, and the Wald z for $\theta_0 = 0.1$. Then compute the LR statistic $2[\ell(\hat{\theta}) - \ell(0.1)]$. Now repeat with *one* success in twenty. Compare the two rulers in each case with lecture's exact rejection-rate table at $n = 20$. Which ruler do you trust here, and why?

8 Session information

```
sessionInfo()
#> R version 4.5.2 (2025-10-31)
#> Platform: aarch64-apple-darwin20
#> Running under: macOS Tahoe 26.6.2
#>
#> Matrix products: default
#> BLAS: /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.frame
#> LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;
#>
#> locale:
#> [1] C.UTF-8/C.UTF-8/C.UTF-8/C/C.UTF-8/C.UTF-8
#>
#> time zone: America/New_York
#> tzcode source: internal
#>
#> attached base packages:
#> [1] stats      graphics  grDevices  utils      datasets  methods   base
#>
#> other attached packages:
#> [1] ggplot2_4.0.2 dplyr_1.2.0
#>
#> loaded via a namespace (and not attached):
#> [1] vctrs_0.7.1      cli_3.6.5        knitr_1.51       rlang_1.1.7
#> [5] xfun_0.56        otel_0.2.0       generics_0.1.4   S7_0.2.1
#> [9] jsonlite_2.0.0   labeling_0.4.3   glue_1.8.0       isoband_0.3.0
#> [13] htmltools_0.5.9  tinytex_0.58     scales_1.4.0     rmarkdown_2.30
#> [17] grid_4.5.2       evaluate_1.0.5   tibble_3.3.1     fastmap_1.2.0
#> [21] yaml_2.3.12      lifecycle_1.0.5  compiler_4.5.2   RColorBrewer_1.1-3
#> [25] pkgconfig_2.0.3  farver_2.1.2     digest_0.6.39    viridisLite_0.4.3
#> [29] R6_2.6.1         tidyselect_1.2.1 pillar_1.11.1     magrittr_2.0.4
#> [33] withr_3.0.2      tools_4.5.2      gtable_0.3.6
```